

LibreChat Project – Work Documentation

Overview

This document summarizes all the work completed in the LibreChat project, including repository setup, deployment attempts, feature integrations, and the development of an Admin Portal for user management.

1. Repository Cloning (Open-Source Setup)

Task Performed:

- Cloned the official open-source LibreChat repository from GitHub: <https://github.com/danny-avila/LibreChat>.

Details:

- Set up the development environment after cloning.
 - Installed required dependencies and configured necessary environment variables.
 - Understood the core structure of the project, including server, client, API integrations, and configuration files.
-

2. Deployment Attempt on Railway

Task Performed:

- Attempted to deploy the cloned LibreChat project on **Railway.app**.

Details:

- Configured Railway project settings for Node.js, backend, and environment variables.
 - Attempted to connect Railway PostgreSQL / MongoDB (as required by LibreChat).
 - Faced initial issues related to environment variables, build commands, and external API dependencies.
 - Documented deployment logs for debugging.
-

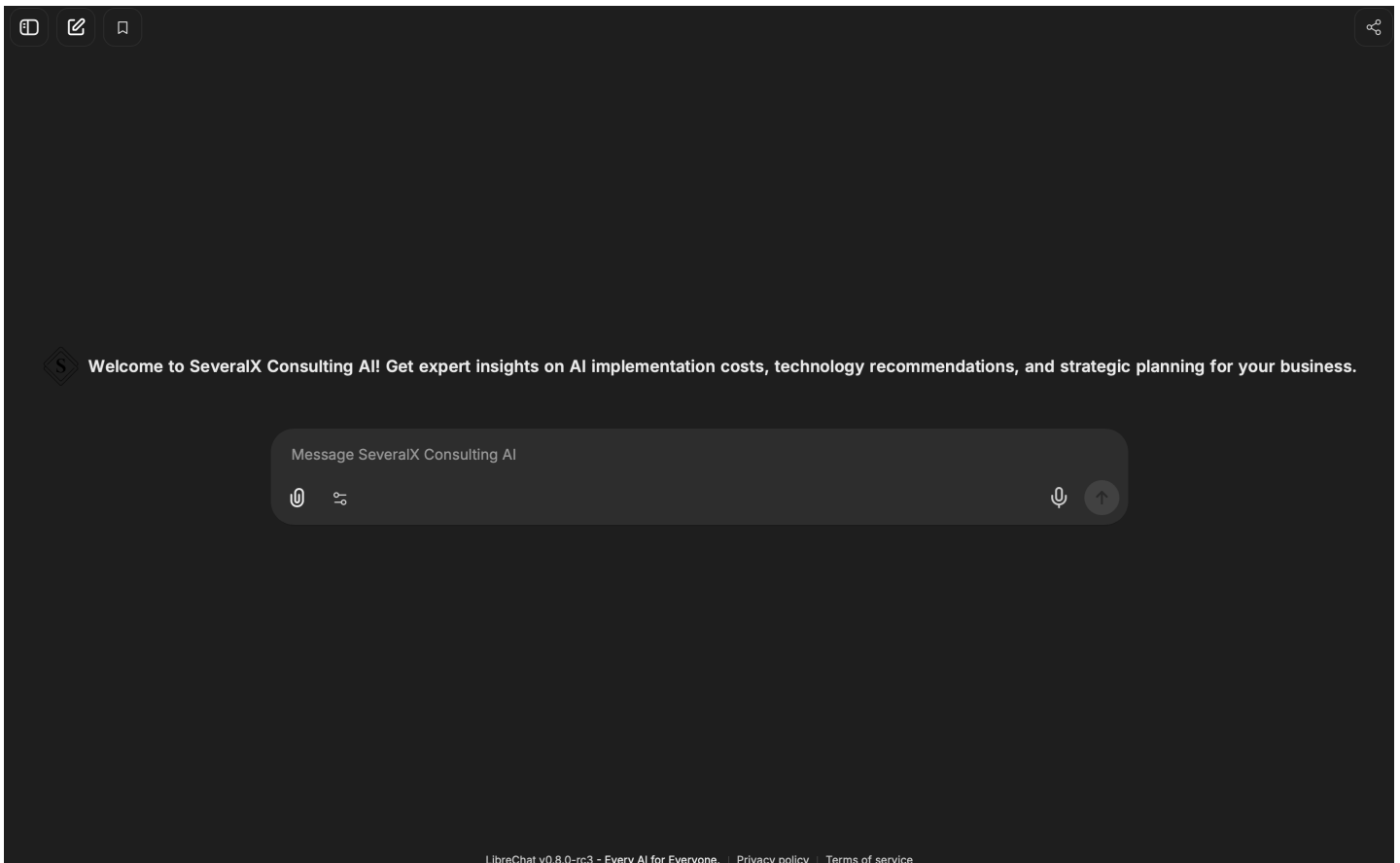
3. Working on AI Agents Connection

Task Performed:

- Started integrating and configuring **AI agents** within LibreChat (librechat.yaml).

Details:

- Explored how LibreChat handles multiple AI model providers (e.g., OpenAI, Azure, Anthropic, HuggingFace).
- Added/customized configuration for AI agents inside the agents or services module.
- Built logic to allow users to interact with custom AI workflows.
- Implemented test prompts and ensured AI agents respond correctly



4. MCP Server Integration

Task Performed:

- Integrated **Model Context Protocol (MCP)** server into the LibreChat backend.

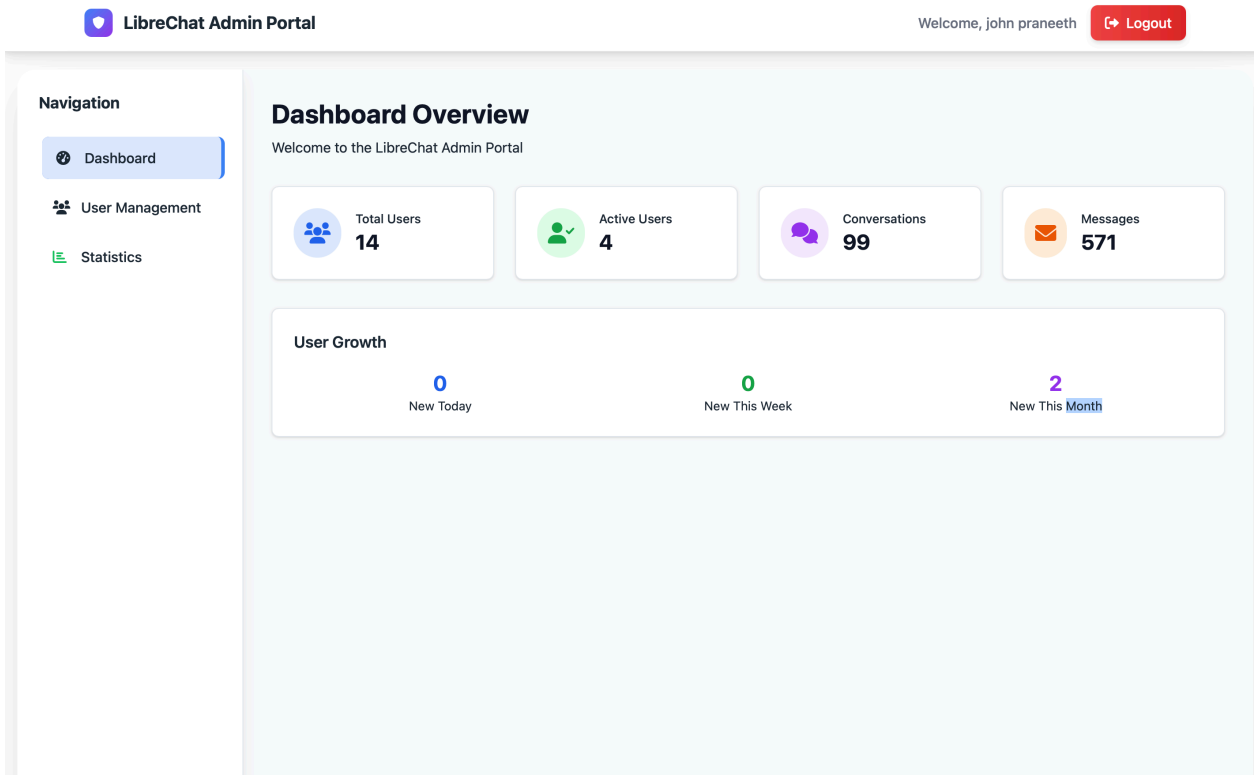
Details:

- Understood how MCP enables tool-calling, function execution, or external resource connections.
 - Connected the LibreChat backend with MCP server endpoints.
 - Wrote integration logic for:
 - Session handling
 - Context passing
 - Tool invocation
 - Implemented test cases to verify MCP server communication.
-

5. Admin Portal for User Management

Task Performed:

- Designed and developed a new **Admin Portal** for managing users in the LibreChat system.



Details:

- Built a dedicated admin dashboard UI.
 - Implemented role-based access (Admin vs Normal User).
 - Added features for:
 - Viewing user details
 - Editing user profiles
 - Activating/deactivating users
 - Resetting user configurations
 - Integrated API routes for admin actions.
 - Ensured secure access using JWT / session tokens.
-

6. Configuration System Overview

LibreChat uses a **hierarchical configuration system** that combines multiple layers of settings to ensure flexibility across development, production, and admin-controlled environments.

6.1 Environment Variables (.env)

Environment variables are used for: - Sensitive credentials (API keys, secrets) - Deployment-specific values - Authentication settings - Rate limiting and security controls - Third-party integrations

These environment variables are typically stored in a .env file and override other configuration sources when loaded.

6.2 LibreChat YAML Configuration (librechat.yaml)

The librechat.yaml file defines: - AI model providers (OpenAI, Anthropic, Azure, Google, etc.) - Endpoint routing - Model-specific parameters (temperature, max tokens, etc.) - Enabled/disabled features - Agent configurations - Plugin/tool integrations

This file acts as the application's primary **feature and model configuration layer**.

6.3 Database Configuration (MongoDB)

At runtime, LibreChat also stores key configuration details in MongoDB. These include: - User-specific settings - Feature flags - Admin-controlled global configuration - Runtime session data - Audit information

This layer allows the system to update configuration dynamically without redeployment.

7. Environment Variables Used in LibreChat

Core Application Environment Variables

Below are the key environment variables configured during the project setup and deployment:

VARIABLE	DESCRIPTION
ALLOW_EMAIL_LOGIN	Enables or disables login using email/password.
ALLOW_REGISTRATION	Allows or blocks new user account registration.
ALLOW_SOCIAL_LOGIN	Enables login through social providers.
ALLOW_SOCIAL_REGISTRATION	Allows new users to register using social login.
ANYSCALE_API_KEY	API key for Anyscale model endpoints.
APP_TITLE	Title displayed for the application UI.
BAN_VIOLATIONS	Threshold of violations before a user is banned.
CHECK_BALANCE	Enables balance checks for API usage.
CONFIG_PATH	Path to the application's config file.
CREDS_IV	Initialization vector for encrypted credentials.
CREDS_KEY	Encryption key for stored credentials.
DEBUG_LOGGING	Enables detailed debug logging.
DEBUG_PLUGINS	Enables plugin-level debug logging.
ENDPOINTS	Configuration of model API endpoints.
FIREWORKS_API_KEY	API key for Fireworks.ai services.
GOOGLE_CALLBACK_URL	OAuth callback URL for Google login.
GOOGLE_KEY	Google OAuth client ID.

VARIABLE	DESCRIPTION
GOOGLE_MODELS	List of enabled Google models.
HOST	Hostname/IP the server runs on.
JWT_REFRESH_SECRET	Secret for generating refresh JWT tokens.
JWT_SECRET	Secret for generating access JWT tokens.
MEILI_HOST	URL of the MeiliSearch server.
MEILI_MASTER_KEY	Admin key for MeiliSearch.
MEILI_NO_ANALYTICS	Disables MeiliSearch analytics.
MESSAGE_USER_MAX	Max messages allowed per user per window.
MESSAGE_USER_WINDOW	Time window for user rate-limiting.
MESSAGE_VIOLATION_SCORE	Violation score per blocked message.
MONGO_URI	MongoDB connection string.
OPENAI_API_KEY	API key for OpenAI services.
OPENAI_MODERATION	Enables OpenAI moderation API.
OPENAI_TITLE_MODEL	Model used for conversation titles.
OPENROUTER_KEY	API key for OpenRouter.
RAG_API_URL	URL of the RAG backend API.
REFRESH_TOKEN_EXPIRY	Expiration time for refresh tokens.
SEARCH	Toggles search features or provider.
SESSION_EXPIRY	Session expiration duration.
TITLE_CONVO	Enables auto-generated conversation titles.
TOGETHERAI_API_KEY	API key for Together.ai models.
ALLOWED_CORS_ORIGINS	List of allowed CORS origins.

VARIABLE	DESCRIPTION
COOKIE_DOMAIN	Domain used for cookies.
DOMAIN_CLIENT	Frontend domain.
DOMAIN_SERVER	Backend domain.
NOTION_API_KEY	API key for Notion API.
NOTION_API_TOKEN	Access token for Notion integrations.
RAILWAY_DOCKERFILE_PATH	Dockerfile path for Railway builds.
LIMIT_MESSAGE_IP	Enables IP-based rate limiting.
MESSAGE_IP_MAX	Max messages allowed per IP.
MESSAGE_IP_WINDOW	Time window for IP rate-limiting.
LIMIT_MESSAGE_USER	Enables user-based rate limiting.
LIMIT_CONCURRENT_MESSAGES	Enables concurrent message limiting.
CONCURRENT_MESSAGE_MAX	Max concurrent messages allowed.
CONCURRENT_VIOLATION_SCORE	Violation score for concurrency abuse.
LOGIN_VIOLATION_SCORE	Violation score for login abuse.
REGISTRATION_VIOLATION_SCORE	Violation score for registration abuse.
BAN_DURATION	Duration of ban.
BAN_INTERVAL	Interval used to evaluate bans.
REGISTER_MAX	Max registrations allowed per window.
REGISTER_WINDOW	Registration rate limit window.
LOGIN_MAX	Max login attempts allowed.
LOGIN_WINDOW	Login attempt rate limit window.

8. Additional Environment Variables for Admin Portal

The Admin Portal required the following environment variables to enable secure access and configuration management:

- AUTO_RESTART_ON_CRITICAL_CHANGE
 - CONFIG_CHANGE_DETECTION
 - GRACEFUL_RESTART_TIMEOUT
 - JWT_SECRET
 - MONGO_URI
 - PORT
 - SUPER_ADMIN_EMAIL
 - SUPER_ADMIN_PASSWORD
-

9. Railway Deployment Architecture

The project deployment on Railway uses a multi-service architecture, with each component containerized and connected through environment variables, shared volumes, and API endpoints.

9.1 Docker Build Strategy (dockerfile.multi)

LibreChat in this project uses a **custom multi-stage Docker build file** named `dockerfile.multi` instead of the default `Dockerfile`.

Why?

- Multi-stage builds reduce final image size.
- Allows separate build stages for client, server, agents, and admin.
- Ensures optimized production builds.

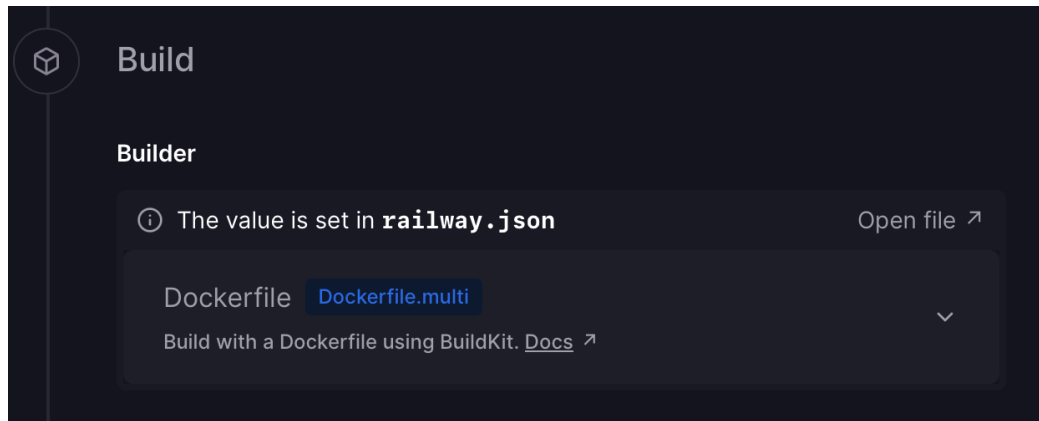
How Railway Detects It

Railway normally looks for a default file named **Dockerfile**, but in this project we explicitly instruct Railway to use **dockerfile.multi**.

railway.json Configuration

The file `railway.json` defines the build settings, including the specific Dockerfile to use: - Overrides default build behavior. - Points Railway to `dockerfile.multi` for building the LibreChat service.

This guarantees Railway uses the correct custom build pipeline.



9.2 librechat.yaml (Core Configuration File)

The `librechat.yaml` file is used for: - Endpoint configurations - AI model provider settings - Model parameters (temperature, maxTokens, penalties, etc.) - Feature toggles (RAG, tools, image generation, memory, etc.) - Agent workflows - Authentication rules and UI configuration

This file acts as the **primary configuration source** for all model behaviors and LibreChat runtime logic.

9.3 Admin Instance – Root-Level Docker Auto-Detection

The **Admin Portal** instance uses a different deployment approach:

Root-Directory Dockerfile

- A Dockerfile placed at the **root directory** of the admin source code.
- We need to add the root directory configuration so that admin instance will admin portal dockerfile instead of librechat dockerfile.
- Railway automatically detects and uses this Dockerfile.

- No need for railway.json override.

Purpose

- Admin portal is treated as an independent micro-service.
 - Allows separate deployment pipeline from the main LibreChat client/server.
 - Makes updates modular and isolated from the main system.
-

9.4 Components in Railway

VectorDB

- Stores embeddings for semantic search and RAG workflows.
- Status: Sleeping since 2 weeks ago.
- Persistent volume attached.

RAG API

- Custom Retrieval-Augmented Generation API used by LibreChat.
- Communicates with VectorDB.
- Status: Sleeping since 2 weeks ago.

MongoDB

- Primary database storing users, conversations, configuration, and runtime data.
- Connected to both **Admin Portal** and **LibreChat**.
- Deployed via Docker image with a persistent volume.

Meilisearch

- Search engine service used for fast document indexing and retrieval.
- Integrated with LibreChat for search features.
- Status: Sleeping since last week.

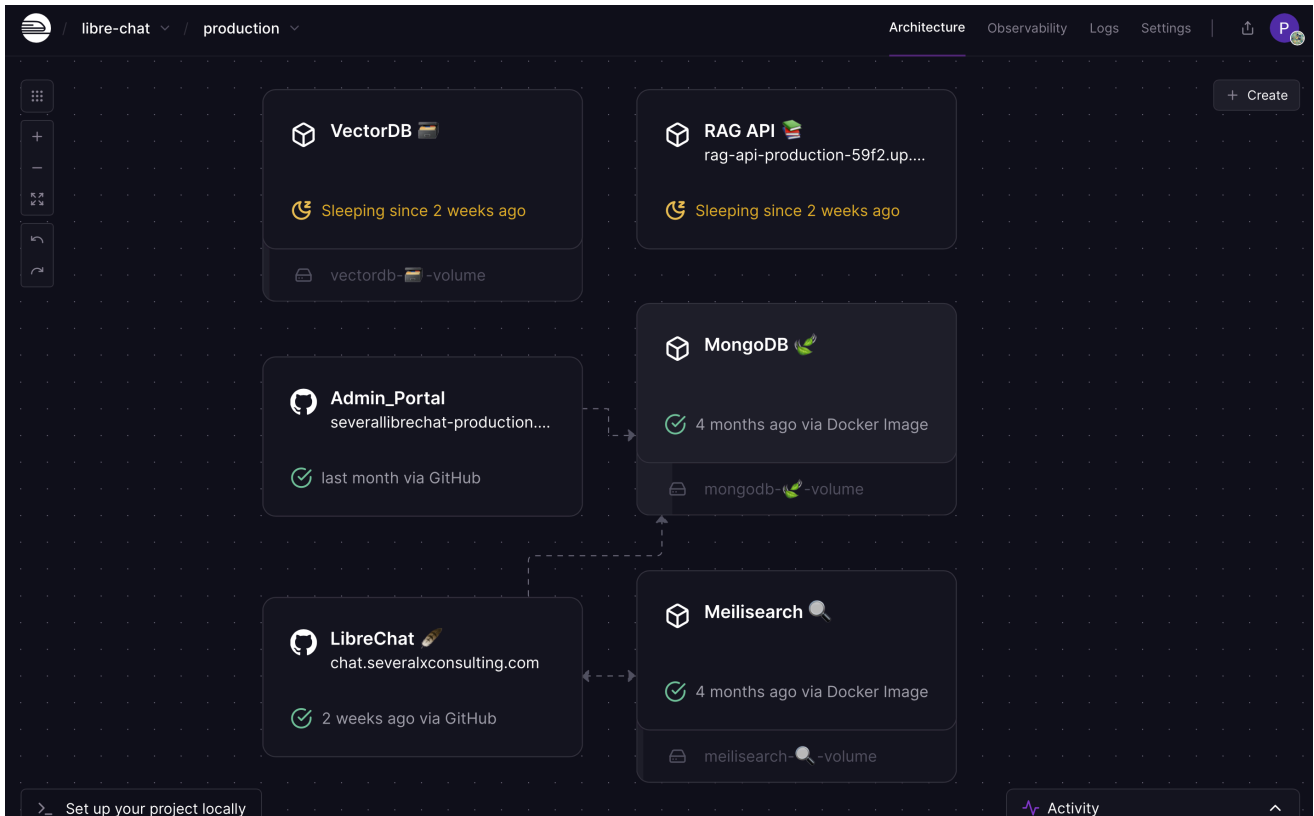
Admin Portal

- Manages users, system configuration, and admin-level controls.
- Connected to **MongoDB**.
- Last deployed via GitHub last month.

LibreChat Application

- Main chat application used by users.
- Hosted at: chat.severalxconsulting.com
- Communicates with:
 - **Admin Portal** (for privileged actions)
 - **MongoDB** (storing conversations, accounts)

- **Meilisearch** (search indexing)
- **RAG API** (retrieval of knowledgebase data)



11. Final Summary

This documentation outlines the major tasks completed in enhancing and customizing the LibreChat project. Work included initial setup, deployment attempts, integration of AI agents and MCP server, and creation of a feature-rich Admin Portal. Further improvements can be added based on project requirements.